EveryRoot

Learning Chess Moves and Search Allocation

Pedro Teles · Architecture research note · September 2026

1. What we are building

Chess has been a serious problem for artificial intelligence since the earliest computers. It remains an unusually clear setting in which to study a practical question: how much calculation does a good decision require? Strong engines extract enormous value from search. We want to make more of that work reusable, and to learn when further calculation is worth its cost.

EveryRoot brings these questions into the same training loop. A learned policy chooses a move; alpha-beta search evaluates its consequences; the evaluation becomes reinforcement-learning feedback. The completed search also supplies a best reply at the position it searched. That reply becomes an imitation target for the same policy when it acts from that position. A separate learner controls how much search to perform.

The central idea is that **one search can teach two chess decisions, while a second agent learns the cost of obtaining those lessons**. The score judges the preceding move. The recommended reply teaches the next decision. Search becomes a source of reusable knowledge, and the amount of computation used to produce that knowledge becomes a learned decision too.

Our aim is an engine capable of competing on playing strength while requiring less computation. Better use of each search could reduce the cost of teaching the player. Better allocation could direct further calculation towards positions where it materially improves the evaluation. Together, these capabilities could support a practical chess-playing and analysis application with lower operating costs.

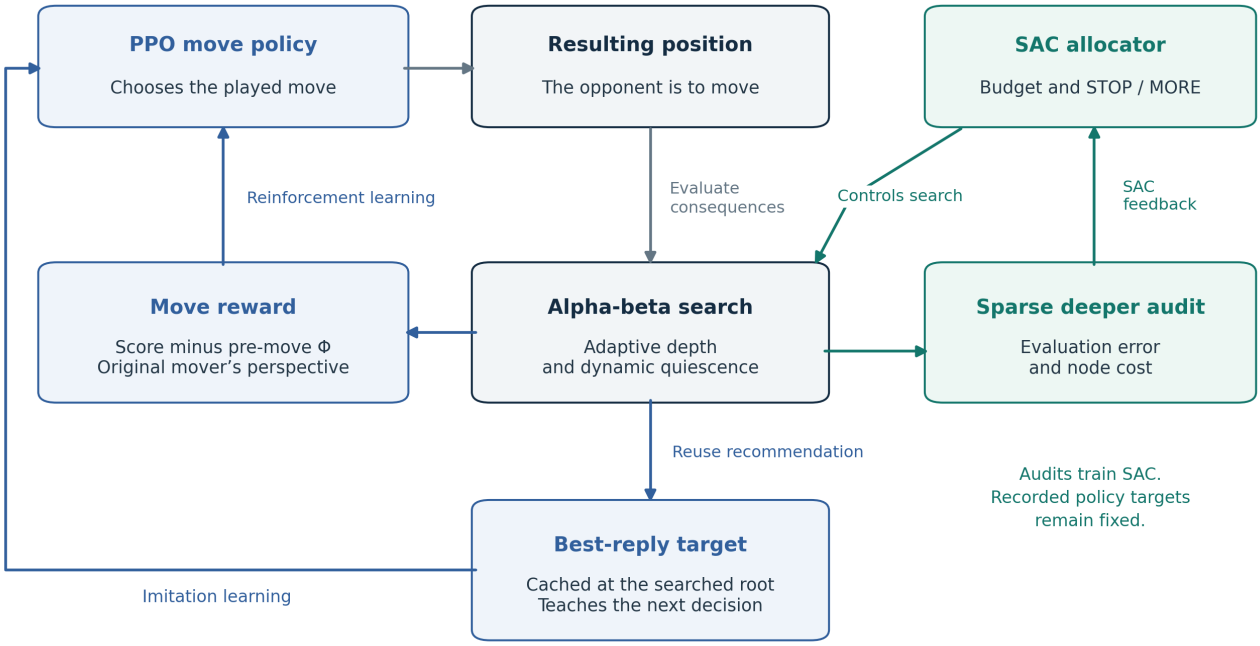
The hypothesis

Learning move selection from alpha-beta feedback and learning the effort spent on that feedback can improve the relationship between playing strength and computational cost. The proposed advance is the particular integration of local reinforcement learning, search imitation and adaptive allocation. PPO, SAC and learning from tree search are established methods and are credited throughout this note. [1-6]

What this note establishes

This document describes the implemented architecture and its research rationale. It presents an engineering and research contribution, with the intended gains stated as hypotheses. Competitive strength, economic savings and transfer to other engines remain outcomes to establish. The development programme aims to scale self-play towards 20 million games.

2. The learning flow



One shared chess policy. A separate learner controls the cost of search.

One policy, two lessons from the searched position

At position s, the controller prepares an initial compute allocation. PPO receives the board and the planned compute context, then selects the actual move a. Native alpha-beta evaluates the resulting position s'. Its completed score is expressed from the original mover's perspective and used to calculate that move's reward.

The search at s' also finds a recommended reply b for the player to move there. When the shared policy next acts from s', b is available as an imitation target. A root key, ply check and legality check keep that label attached to the position it describes. Because the same policy plays both colours, the search can teach the reply as well as judge the preceding move.

For example, after White selects a move, the resulting search considers Black's replies. Its score feeds back to White's decision. Its recommended Black reply teaches the policy at Black's position. It is never presented as an alternative White move.

Allocation and audit feedback

During search, the controller can grant more nodes or stop. A sparse deeper continuation supplies additional evidence for SAC about whether stopping left a substantial evaluation error. This audit is separate from PPO's recorded reward and cached teacher label. Later audit work does not retrospectively alter either of them.

3. Learning what to play

EveryRoot uses one residual move policy, with approximately 18M parameters, shared across both colours. Legal-move masking restricts the action space. The policy receives a side-to-move representation and an initial compute-context scalar. Completed depth and the search tree are not retrospectively added to the input from which an earlier action was sampled.

A local reward from tactical search

The played move receives the following reward, with both scores in the mover's frame:

$$r = 0.1 \times [AB_{adaptive}(s', mover) - \Phi_{static}(s, mover)]$$

Here Φ is the static evaluation before the move, and AB is the backed-up evaluation of the resulting position under adaptive alpha-beta and dynamic quiescence. This makes tactical consequences visible to the learner through each judged move. The reward is a local evaluation difference; it is not a calibrated win probability or exact regret relative to the best legal move.

The policy update uses a clipped PPO likelihood-ratio surrogate, taking this reward directly as its advantage signal. This is a policy-only, contextual-bandit-style use of PPO: there is no value head, GAE or discounted full-game return in the current policy objective. The original PPO method supplies the update machinery; the local search reward is the design choice made here. [1]

A concrete move to learn from

The cached best move from a completed search is also a supervised target at that search's own root. Its negative log probability supplies an auxiliary imitation term in the ordinary policy update. In the current configuration its coefficient is 0.10; rows without a valid teacher label contribute zero to that term.

$$L_{policy} = L_{clipped\_PPO} - \beta H(\pi) + 0.10 L_{imitation}$$

This supplements an evaluation of the action actually taken with a specific recommended action at another correctly aligned training state. It is a form of search imitation, related to the broader approach of learning policies from planning. It does not require a probability distribution over all root moves, and it does not claim an exact score for a pruned alternative. [4]

Why the combination is useful

The scalar reward preserves learning from sampled decisions. Imitation makes use of an explicit recommendation the engine already computed. Together they offer more focused teaching than a scalar reward alone, without adding a teacher search or policy forward pass for those labels. Constructing the target and adding its loss still has a small computational cost.

This is an intended sample-efficiency benefit, not a guarantee of faster convergence. The two losses can disagree, and the search teacher can miss tactics. Giving the network a teacher label improves the information available for learning; it does not make that information infallible.

4. Learning how much to calculate

The SAC controller treats computation as an action with a cost. It sees a board embedding and search telemetry, and produces a continuous action. That action sets an initial node allocation or requests further search. During continuation, a non-positive proposal means STOP, subject to minimum-depth and curriculum rules; positive proposals map logarithmically to additional nodes.

Learn allocation from a working teacher

The controller first imitates allocation rules derived from Stockfish time management. A gradual handoff blends these proposals with SAC's own decisions, allowing the learned controller to take responsibility progressively. This acknowledges the practical value of an existing allocator while creating a route to learning a position-dependent replacement. The source explicitly attributes the copied allocation equations to Stockfish. [7]

Reward accuracy gained against computation spent

Sparse deeper continuations supply a reference evaluation. For audited search transitions, the controller receives credit when further search reduces disagreement with that reference and pays for the added nodes. Stopping with residual error is also penalised. The central structure is:

$$r_{compute} = \text{evaluation-error reduction} - \lambda \times \text{normalised node cost}$$

The implementation uses a Huber-scaled evaluation error, reward and cost clipping, and an additional residual-error penalty at the final audited transition. A learned price λ adjusts the balance between evaluation fidelity and node expenditure relative to the teacher's error target. Quiescence nodes count towards cost. Wall-clock time is not the direct reward variable.

SAC provides an actor, twin critics, replay and entropy regularisation for this allocation problem. Its entropy coefficient is learned during reinforcement-learning updates; it is distinct from the teacher-handoff weight. These mechanisms follow established SAC work. [2,3]

What is separated, and why

SAC learns through its own networks and replay. Its audit evidence does not overwrite the reward used by PPO for an already collected move. This separates the effort controller's learning problem from the player's stored teaching signal, while allowing future allocations to change as SAC improves.

The deeper reference is a continuation of the same engine, with the same evaluator's limitations. SAC therefore learns fidelity to that reference, not an oracle measure of chess truth. Conventional branch pruning, reductions, transposition tables and dynamic quiescence remain in the native engine. The learned component controls budgets and stopping.

5. Contribution to chess-engine research

Modern engine research already combines search and learning. Stockfish uses neural evaluation within a selective search architecture, while Leela Chess Zero uses policy/value networks to guide tree search. AlphaZero established a powerful route through self-play and search, and Expert Iteration explicitly studies the relationship between planning and learning to generalise its results. [4-8]

EveryRoot explores a particular arrangement within this field: a policy learns from the local consequences of its chosen moves and from a cached recommendation at the next searched root, while a separate reinforcement-learning controller learns the effort spent producing those teaching signals. Its current search leaves use a classical static evaluator. The proposed contribution is this integrated learning flow, rather than a new claim to the underlying PPO, SAC or distillation methods.

Features that make the design worth pursuing

- **Two useful teaching signals from one completed search.** An evaluation of the played move and a recommended reply can both train the shared policy. Correct root alignment makes that reuse possible.
- **A learned budget for the teacher.** Search cost becomes a decision conditioned on the position and search progress. This connects the quality of instruction with the cost of obtaining it.
- **A gradual transition from handcrafted allocation.** Teacher imitation supplies an initial operating regime, followed by an annealed handoff to learned decisions.
- **Separate player and controller objectives.** Move learning and compute learning can develop without using later controller audits to revise earlier policy targets.
- **An implementation built to keep work moving.** Batched neural inference and native CPU searches overlap through an asynchronous reservoir of games. Individual slow searches need not hold up an entire policy batch.

These choices make the research question concrete: can the policy retain and generalise useful tactical knowledge while the controller reduces computation that contributes little to its evaluator? A successful answer would extend work on search-guided learning with evidence for jointly learning a player and the allocation of its teaching effort.

What an improvement would mean

A meaningful engine advance would be stronger play within the same overall compute budget, or comparable play with less computation. Node savings alone would be incomplete: neural inference, controller calls and system overhead also cost time and energy. Similarly, better imitation of a teacher is valuable only insofar as it produces useful chess decisions.

EveryRoot is a working research prototype with a specific approach to improving chess strength per unit of computation. Establishing that improvement would turn the proposed integration into an empirical contribution.

6. What success would make possible

A competitive engine with lower operating costs

If a move policy retains tactical knowledge from previous searches, it may select better moves across related positions. If the controller also learns which searches deserve more work, the system could spend its computational budget where it matters most. The intended outcome is a better balance between chess strength, response time and running cost.

That balance matters for a practical application. A chess-playing and analysis service must serve repeated requests within a hardware and latency budget. Comparable analysis at lower cost could make longer sessions and broader access affordable; stronger play at the same cost could improve the quality of the product. These are product opportunities that follow from the research goal, not claims about an app already on the market.

A useful allocation component in its own right

The learned allocator also has a possible application beyond the current PPO player. An engine that lets alpha-beta select the move could use a neural controller for its search budget and stopping decisions. This would preserve conventional move selection while investigating learned allocation of effort.

Such a transfer would require adapting the controller's inputs, reward and reference signals to the host engine. It should not be assumed that a controller trained to evaluate PPO-selected children already knows how best to manage a full move-selection search. The component is a plausible second development path because allocation is explicitly separated in the architecture.

The programme

We intend to scale the self-play programme towards 20 million games while retaining the direct learning loop. The compute requirement includes both neural policy learning on GPUs and substantial CPU capacity for native search. The present implementation uses PyTorch and native CPU code; it does not currently use the Emmy compiler.

The central objective is a convincing chess engine with a practical cost of operation. The application follows from that capability. The public materials document the architecture and its rationale; engine source and trained weights are outside this release.

Current limits of the approach

Adaptive horizons can produce different rewards for the same position and move. The planned compute input does not reveal all future controller decisions or search outcomes. The policy's local objective also does not propagate the final game result back through every earlier action. These are properties of the present learning problem, and neither imitation nor more training automatically removes them.

The research case is nevertheless concrete: make the search teach more, and learn the effort spent doing that teaching. If this delivers stronger chess per unit of computation, it would be useful both as an engine design and as a way to control the cost of search-based learning.

References and implementation provenance

1. Schulman, J., Wolski, F., Dhariwal, P., Radford, A. and Klimov, O. (2017). *Proximal Policy Optimization Algorithms*. arXiv:1707.06347.

2. Haarnoja, T., Zhou, A., Abbeel, P. and Levine, S. (2018). *Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor*. arXiv:1801.01290.

3. Haarnoja, T. et al. (2018). *Soft Actor-Critic Algorithms and Applications*. arXiv:1812.05905.

4. Anthony, T., Tian, Z. and Barber, D. (2017). *Thinking Fast and Slow with Deep Learning and Tree Search*. arXiv:1705.08439.

5. Silver, D. et al. (2017). *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*. arXiv:1712.01815.

6. Leela Chess Zero contributors. *Technical Explanation of Leela Chess Zero*. Official project documentation. Accessed September 2026.

7. Stockfish contributors. *Time management implementation*, `src/timeman.cpp`. Official source. EveryRoot's allocation-teacher source records upstream revision `47be34c55fbc86079cba57b9ad6955e6fe0bdff9`. The recorded revision is implementation provenance, not a claim that this report independently reproduced that upstream revision. Stockfish is distributed under GPLv3. Upstream licence.

8. Stockfish contributors. *Stockfish*. Official project repository. Engine architecture and NNUE provenance. Accessed September 2026.

Scope of the implementation description

The architecture was checked against the EveryRoot build containing search imitation, SAC annealing and the updated diagnostics. The principal implementation locations are `single_policy.py` (shared policy), `single_ppo.py` (clipped objective and imitation), `single_selfplay.py` (execution order and teacher alignment), `alpha_beta_reward.py` (adaptive judgement and audits), `compute_sac.py` (controller), and `sf_compute_teacher.py` (allocation teacher).

Earlier design discussions included different ordering and state-input proposals. This note describes the implemented choose-then-evaluate loop, with completed search recommendations used as training labels. It does not describe a search-first actor or an all-move value-distillation system.

This is an architecture disclosure, not a reproducible software release: source and weights are not included. Upstream attribution is retained, and the report does not grant a licence for any third-party implementation. Copyright and rights for the authored document are specified in the accompanying RIGHTS.md.